

K-Nearest Neighbors (KNN)

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezf@udesa.edu.ar y lo revisamos.

1. Introducción a K-Nearest Neighbors

K-Nearest Neighbors (KNN) es uno de los algoritmos de machine learning más simples e intuitivos. Se basa en el principio de que “las cosas similares tienden a estar cerca”. En lugar de aprender un modelo explícito durante el entrenamiento, KNN almacena todos los datos de entrenamiento y hace predicciones basándose en la proximidad de nuevos puntos a los datos existentes.

El algoritmo funciona de la siguiente manera:

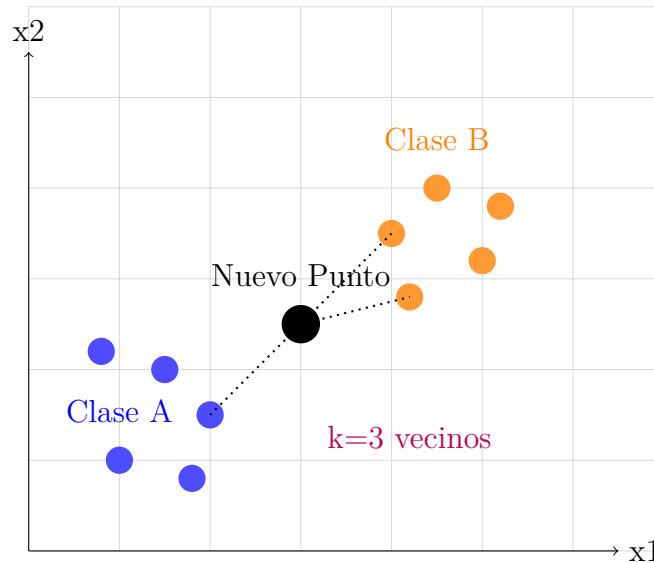
1. Cuando llega un nuevo punto a clasificar o predecir
2. Calcula la distancia a todos los puntos de entrenamiento
3. Selecciona los k puntos más cercanos (vecinos)
4. Para clasificación: vota por la clase mayoritaria entre los vecinos
5. Para regresión: promedia los valores objetivo de los vecinos

Las características principales de KNN son:

- Es un algoritmo “lazy” (vago): no entrena un modelo explícito
- Es no paramétrico: no hace suposiciones sobre la distribución de los datos
- Es sensible a la escala de las variables
- Su rendimiento depende crucialmente del valor de k y la métrica de distancia

2. Visualización del Concepto

Para entender cómo funciona KNN, consideremos un ejemplo visual con datos en 2D:



En este caso tenemos una separación muy clara de clases, pero siempre va a pasar de que estén bastante mas mezclados. Con $k=3$, los tres vecinos más cercanos al punto negro son: dos puntos naranjas (Clase B) y un punto azul (Clase A). Por tanto, la predicción sería Clase B (2 votos vs 1 voto).

3. KNN para Clasificación

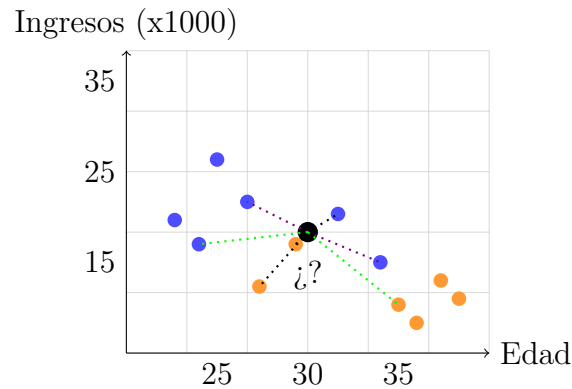
En problemas de clasificación, KNN asigna una clase al nuevo punto basándose en las clases de sus k vecinos más cercanos.

3.1. Algoritmo de Clasificación

1. Dado un nuevo punto x_{nuevo} a clasificar calcular la distancia de x_{nuevo} a todos los puntos de entrenamiento
2. Seleccionar los k puntos con menor distancia
3. Contar los votos de cada clase entre estos k vecinos y asignar la clase con mayor número de votos

3.2. Ejemplo Numérico

Consideremos un dataset simple con dos variables y dos clases: puntos azules (Aprobado) y puntos naranjas (Rechazado).



El efecto del parámetro k en la predicción:

- **$k=3$ (líneas negras):** 1 aprobado, 2 rechazado $\rightarrow$ Predicción: Rechazado.
- **$k=5$ (líneas violetas + negras):** 3 aprobados, 2 rechazados $\rightarrow$ Predicción: Aprobado.
- **$k=7$ (líneas verdes + violetas + negras):** 4 aprobados, 3 rechazados $\rightarrow$ Predicción: Aprobado.

A medida que el k aumenta la predicción puede ir cambiando. En general se vuelve más conservadora porque considera más puntos lejanos. Pregunta: ¿Qué pasa si $K=N$? Esto está contestado justo abajo pero piensenlo un segundo.

3.3. Efecto del Parámetro k

El valor de k tiene un impacto significativo en las predicciones:

- **$k = 1$:** Alta varianza, propenso a overfitting, sensible al ruido
- **k pequeño (3-5):** Fronteras de decisión complejas, captura patrones locales
- **k grande:** Fronteras de decisión más suaves, reduce overfitting pero puede causar underfitting
- **$k = n$ (total de datos):** Siempre predice la clase mayoritaria

3.4. Selección del Parámetro k

La elección del parámetro k es crucial para el rendimiento del algoritmo. Esto puede afectar directamente la precisión de las predicciones y la capacidad del modelo para generalizar.

- **Regla empírica:** $k = \sqrt{n}$ donde n es el número de observaciones.
- **Cross-validation:** Probar diferentes valores y elegir el que minimice el error.
- **Grid search:** Búsqueda exhaustiva en un rango de valores.

Algunas consideraciones importantes que tenemos que siempre tener en cuenta son:

- k impar evita empates en clasificación binaria.
- k muy pequeño provoca overfitting.
- k muy grande provoca underfitting.
- considerar el balance de clases. Si una clase es muy mayoritaria sobre la otra cuanto más grande sea el k más probable es que el modelo prediga la clase mayoritaria.

4. KNN para Regresión

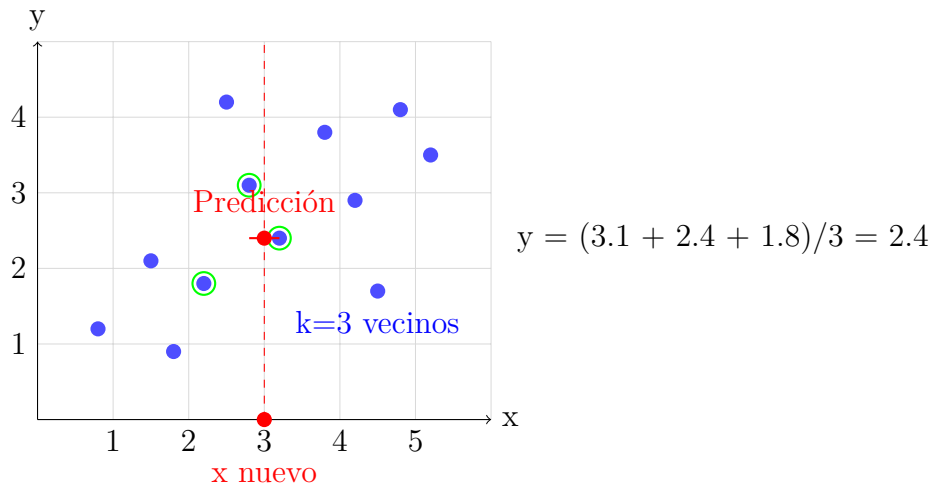
En problemas de regresión, KNN predice un valor numérico promediando los valores objetivo de los k vecinos más cercanos.

4.1. Algoritmo de Regresión

1. Dado un nuevo punto x_{nuevo} para predecir calcular la distancia de x_{nuevo} a todos los puntos de entrenamiento
2. Seleccionar los k puntos con menor distancia
3. Calcular el promedio de los valores objetivo de estos k vecinos y retornar este promedio como predicción

4.2. Ejemplo Visual de Regresión

En este caso tenemos un solo dato, que es el valor de X y la idea es poder predecir Y , como por ejemplo con solo la edad predecir ingresos. Si tuvieramos dos variables, como X e Y entonces predeciríamos un Z , como si teniendo edad e ingresos pudiéramos predecir un score de crédito. Siempre a partir de la o las variables que tenemos predecimos una dimensión más.



4.3. Pesos por Distancia

Hay veces en donde sería lógico darle más importancia a los vecinos más cercanos. Lo que podemos hacer ahí es en lugar de un promedio simple, usar pesos inversamente proporcionales a la distancia:

$$\hat{y} = \frac{\sum_{i=1}^k w_i \cdot y_i}{\sum_{i=1}^k w_i}$$

donde $w_i = \frac{1}{d_i}$ y d_i es la distancia al vecino i-ésimo.

5. Métricas de Distancia

La elección de la métrica de distancia es crucial para el rendimiento de KNN. Esto se debe a que:

- **Determina qué puntos considera "similares"**: Si usás la métrica incorrecta, vas a buscar vecinos que en realidad no son parecidos a tu punto.

- **Afecta la predicción final:** Diferentes métricas pueden darte predicciones completamente distintas para el mismo punto.
- **Es sensible a la escala de las variables:** Una variable con números grandes puede dominar el cálculo si no elegís bien la métrica.
- **Debe coincidir con el tipo de datos:** No podés usar la misma métrica para edades que para colores de auto.

Pensalo así: si querés encontrar personas similares a vos, no vas a medir la distancia de la misma forma si comparás altura + peso (números) que si comparás color de pelo + tipo de música (categorías).

5.1. Distancia Euclidiana

La más común para variables continuas. ¿Cuándo la podemos usar? Cuando tenemos variables con números (como edad, ingresos, altura, peso) y las variables están en la misma escala. Se calcula de la siguiente forma:

$$d(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

Ejemplo práctico: Predecir precio de casa basado en metros cuadrados y número de habitaciones.

- Casa A: 100m², 3 habitaciones, precio \$150,000
- Casa B: 120m², 4 habitaciones, precio \$180,000
- Casa nueva: 105m², 3 habitaciones, precio desconocido

Distancia de la casa nueva a Casa A:

$$d(x_1, x_2) = \sqrt{(105 - 100)^2 + (3 - 3)^2} = \sqrt{25 + 0} = 5,00$$

Distancia de la casa nueva a Casa B:

$$d(x_1, x_2) = \sqrt{(105 - 120)^2 + (3 - 4)^2} = \sqrt{225 + 1} = 15,03$$

Si k=1, elegís la más cercana (Casa A). Predicción: \$150,000

Si k=2, promediás ambas: $(\$150,000 + \$180,000)/2 = \$165,000$

Pero hay un problema: Si una variable tiene números mucho más grandes, domina todo. Si comparás ingresos (en miles) con número de hijos, los ingresos van a mandar.

5.2. Distancia Manhattan

La distancia Manhattan se llama así porque la forma en la que se calcula está inspirada en la forma de desplazarse en una ciudad organizada en cuadrículas como Manhattan. En este tipo de ciudades para desplazarte de una esquina a otra no puedes moverte en diagonal a la “Euclidea” si no que tienes que avanzar calle por calle en direcciones verticales y horizontales. Por ejemplo si estuviéramos en la 1st Ave y la 2nd St y queremos llegar a la 4th Ave y la 6th St deberíamos movernos 3 cuadras en una dirección y 4 en la otra, para un total de 7 cuadras. Con Euclideana sería $\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$ cuadras pero en la vida real no puedes caminar en diagonal atravesando edificios (a menos que seas Spiderman).

Útil cuando las variables tienen interpretaciones diferentes. La vamos a usar con variables que representen cantidades o conteos (como número de clicks o días de vacaciones). También cuando tengamos datos con muchas dimensiones o cuando querés considerar los outliers pero sin que dominen el cálculo (porque la euclidiana les da más peso al elevar esa diferencia al cuadrado).

$$d(x_1, x_2) = \sum_{i=1}^n |x_{1i} - x_{2i}|$$

Ejemplo práctico: Recomendación de productos basada en compras por categoría.

- Cliente A: 5 libros, 2 películas, 3 ropa, le gusta la ciencia ficción
- Cliente B: 3 libros, 4 películas, 1 ropa, le gusta el drama
- Cliente C: 2 libros, 1 película, 4 ropa, le gusta la ciencia ficción
- Cliente nuevo: 4 libros, 2 películas, 2 ropa, género desconocido

Distancia del cliente nuevo a Cliente A:

$$d(x_1, x_2) = |4 - 5| + |2 - 2| + |2 - 3| = 1 + 0 + 1 = 2$$

Distancia del cliente nuevo a Cliente B:

$$d(x_1, x_2) = |4 - 3| + |2 - 4| + |2 - 1| = 1 + 2 + 1 = 4$$

Distancia del cliente nuevo a Cliente C:

$$d(x_1, x_2) = |4 - 2| + |2 - 1| + |2 - 4| = 2 + 1 + 2 = 5$$

Si $k=1$, elegís la más cercana (Cliente A). Predicción: le gusta ciencia ficción

Si $k=3$, elegís los 3 más cercanos: Cliente A (ciencia ficción), Cliente B (drama), Cliente C (ciencia ficción) → Predicción: le gusta ciencia ficción.

5.3. Distancia de Minkowski

La distancia de Minkowski es una generalización de la distancia Euclídea y la distancia Manhattan. La vamos a usar cuando queremos tener flexibilidad en la métrica y poder cambiar un hiperparámetro ajustándolo. El parámetro p te permite penalizar las diferencias entre coordenadas

$$d(x_1, x_2) = \left(\sum_{i=1}^n |x_{1i} - x_{2i}|^p \right)^{1/p}$$

Casos especiales:

- $p = 1$: Distancia Manhattan (suma de diferencias absolutas).
- $p = 2$: Distancia Euclidiana (la diagonal “real”).
- $p = 3, 4, 5$: A medida que p aumenta, las diferencias grandes pesan más (se penalizan más fuerte).
- $p \rightarrow \infty$: Distancia de Chebyshev. Solo considera la diferencia máxima en alguna coordenada. Ejemplo: en un proceso de fábrica en paralelo, el tiempo total lo determina la etapa más lenta (la mayor diferencia porque es el cuello de botella).

Ejemplo práctico: Clasificación de estudiantes basada en notas (del 1 al 10).

- Estudiante A: 10, 7, 6, 9
- Estudiante B: 6, 8, 8, 7
- Con $p=1$: $|10 - 6| + |7 - 8| + |6 - 8| + |9 - 7| = 4 + 1 + 2 + 2 = 9$
- Con $p=2$: $\sqrt{4^2 + 1^2 + 2^2 + 2^2} = \sqrt{16 + 1 + 4 + 4} = \sqrt{25} = 5$
- Con $p=3$: $\sqrt[3]{4^3 + 1^3 + 2^3 + 2^3} = \sqrt[3]{64 + 1 + 8 + 8} = \sqrt[3]{81} = 4,33$
- Con $p=7$: $\sqrt[7]{4^7 + 1^7 + 2^7 + 2^7} \approx \sqrt[7]{16384 + 1 + 128 + 128} = \sqrt[7]{16641} \approx 3,16$
- Con p muy grande (Chebyshev): $\max(|10 - 6|, |7 - 8|, |6 - 8|, |9 - 7|) = 4$

5.4. Distancia de Hamming

Cuando trabajamos con KNN, a veces los datos no son numéricos, sino categóricos o binarios, como género, color, tipo de producto, respuestas sí/no o códigos de longitud fija. En esos casos no tiene sentido medir distancias geométricas como Euclídea o Manhattan, porque no podemos representar atributos como colores o estados en un eje X/Y. La distancia de Hamming es ideal para estos escenarios: mide simplemente cuántas posiciones difieren entre dos instancias, contando cada atributo distinto como 1 y los iguales como 0.

$$d(x_1, x_2) = \sum_{i=1}^n I(x_{1i} \neq x_{2i})$$

donde $I(\cdot)$ es la función indicadora: vale 1 si la condición se cumple y 0 si no.
Ejemplo práctico: Clasificación de clientes basada en características categóricas.

- Cliente A: M, Soltero, Empleado, **Sí** tiene tarjeta.
- Cliente B: F, Casada, Empleada, **No** tiene tarjeta.
- Comparación posición por posición:
 - M vs F $\rightarrow$ diferente (1)
 - Soltero vs Casada $\rightarrow$ diferente (1)
 - Empleado vs Empleada $\rightarrow$ igual (0)
 - Sí tiene tarjeta vs No tiene tarjeta $\rightarrow$ diferente (1)
- Distancia de Hamming = $1 + 1 + 0 + 1 = 3$

Ventaja: Es súper simple: solo contás cuántas características son diferentes. Perfecto para datos que no son números (porque no podríamos plotear en un eje X el color de ojos de una persona por ejemplo)

5.5. Resumen: ¿Cuál elegir?

- Solo números, misma escala $\rightarrow$ Euclidiana.
- Solo números, escalas diferentes $\rightarrow$ Manhattan (o normalizar + euclidiana).
- Solo categorías $\rightarrow$ Hamming.

- **Mezcla de números y categorías** → Combinar métricas o convertir todo a números (con criterio).
- **Querés controlar qué tan “exigente” es** → Minkowski con p que te guste.
- **No estás seguro** → Probá Manhattan, es la más robusta.

6. Normalización de Variables

KNN es muy sensible a la escala de las variables, ya que el cálculo de distancia se ve fuertemente influenciado por aquellas características que tienen valores grandes. Si una variable tiene magnitudes mucho mayores que otra, puede dominar la distancia y, por lo tanto, el resultado de la clasificación o regresión.

6.1. Ejemplo sin Normalización

Consideremos un conjunto de datos de empleados con las siguientes características:

- Edad: 25 a 60 años
- Salario: 30,000 a 100,000 pesos

Si calculamos la distancia euclidiana entre empleados, el salario dominará el resultado debido a su escala mucho mayor, mientras que la edad prácticamente no influirá en la distancia.

6.2. Min-Max Scaling

Una forma de corregir esto es escalar las variables al rango $[0,1]$, usando la fórmula:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}$$

De esta manera, todas las variables quedan en la misma escala, preservando la distribución relativa de los datos.

6.3. Z-Score Standardization

Otra opción es estandarizar las variables a media 0 y desviación estándar 1, usando:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

Esto centra los datos y ajusta la dispersión, evitando que variables con varianzas grandes dominen el cálculo de distancia.

7. Ventajas y Desventajas de KNN

7.1. Ventajas

KNN es un algoritmo fácil de entender y de implementar, ideal para arrancar con aprendizaje supervisado. No necesita asumir ninguna distribución particular de los datos, porque es un método no paramétrico, y se puede usar tanto para clasificación como para regresión. Además, es flexible: se adapta a nuevas observaciones sin tener que reentrenar todo el modelo, y si elegís bien el número de vecinos y la métrica de distancia, puede capturar relaciones complejas entre los datos.

7.2. Desventajas

A pesar de ser simple, KNN puede ser pesado cuando tenés muchos datos, porque calcula la distancia de la nueva observación con todos los puntos del conjunto de entrenamiento. Es muy sensible a la escala de las variables, así que generalmente hay que normalizar o estandarizar antes de usarlo. También puede verse afectado por ruido y valores atípicos que pueden distorsionar la clasificación. La elección de k es clave: un valor mal elegido puede arruinar el desempeño del modelo. Por último, KNN no genera un modelo explícito; toda la info queda en los datos de entrenamiento, lo que a veces hace que sea menos interpretable.

Regla de oro: Si no sabés cuál usar, empezá con Manhattan. Es la más "justa" funciona bien en la mayoría de casos.

8. Curse of Dimensionality

Cuando trabajamos en espacios de alta dimensionalidad, todos los puntos tienden a estar prácticamente a la misma distancia entre sí. Esto hace que el concepto de

“vecino más cercano” pierda sentido, porque ya no hay puntos claramente próximos o lejanos.

8.1. Problema

A medida que aumenta la cantidad de dimensiones, el volumen del espacio crece de manera exponencial y los datos se vuelven mucho más dispersos. Como consecuencia, las distancias entre puntos pierden capacidad de discriminación: los vecinos dejan de ser realmente cercanos, y esto afecta directamente el desempeño de algoritmos basados en distancias, como KNN.

8.2. Soluciones

Para mitigar este problema, se pueden tomar varias estrategias. Una es realizar una selección de características relevantes, quedándose solo con las dimensiones que aportan información útil. Otra opción es aplicar técnicas de reducción de dimensionalidad, como PCA o t-SNE, que condensan la información en menos dimensiones sin perder lo esencial. También se puede recurrir a métricas de distancia especializadas que sean más robustas en espacios de alta dimensión, o aumentar el tamaño del dataset para que los puntos cercanos sigan existiendo y los vecinos mantengan sentido.

9. Implementación en Python

```
1 from sklearn.neighbors import KNeighborsClassifier,
   KNeighborsRegressor
2 from sklearn.preprocessing import StandardScaler # o MinMaxScaler
3 from sklearn.model_selection import cross_val_score,
   train_test_split
4 from sklearn.metrics import classification_report, confusion_matrix
5 from sklearn.metrics import mean_squared_error, r2_score
6
7 # Preparacion de datos
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
   =0.2, random_state=42)
9
10 # Normalizacion
11 scaler = StandardScaler() # o MinMaxScaler()
12 X_train_scaled = scaler.fit_transform(X_train)
13 X_test_scaled = scaler.transform(X_test)
14
```

```

15 # Para clasificacion
16 knn_clf = KNeighborsClassifier(
17     n_neighbors=5,
18     weights='distance', # Pesos por distancia
19     metric='euclidean'
20 )
21
22 # Para regresion
23 knn_reg = KNeighborsRegressor(
24     n_neighbors=3,
25     weights='uniform',
26     metric='manhattan'
27 )
28
29 # Entrenamiento
30 knn_clf.fit(X_train_scaled, y_train)
31 knn_reg.fit(X_train_scaled, y_train)
32
33 # Predicciones
34 y_pred_clf = knn_clf.predict(X_test_scaled)
35 y_pred_reg = knn_reg.predict(X_test_scaled)
36
37 # Validacion cruzada para seleccionar k
38 k_values = range(1, 31)
39 cv_scores = []
40
41 for k in k_values:
42     knn = KNeighborsClassifier(n_neighbors=k)
43     scores = cross_val_score(knn, X_train_scaled, y_train, cv=5,
44                               scoring='accuracy')
45     cv_scores.append(scores.mean())
46
47 optimal_k = k_values[np.argmax(cv_scores)]
48 print(f"Mejor k: {optimal_k}")
49
50 # Evaluacion de clasificacion
51 print("Reporte de clasificacion:")
52 print(classification_report(y_test, y_pred_clf))
53 print("Matriz de confusion:")
54 print(confusion_matrix(y_test, y_pred_clf))
55
56 # Evaluacion de regresion
57 mse = mean_squared_error(y_test, y_pred_reg)
58 r2 = r2_score(y_test, y_pred_reg)
59 print(f"MSE: {mse}")
60 print(f"R-squared: {r2}")

```

10. Casos de Uso y Aplicaciones

10.1. Clasificación

Para clasificación, KNN se utiliza para predecir la categoría a la que pertenece una nueva instancia comparándola con sus vecinos más cercanos. Por ejemplo, se puede clasificar clientes según si es probable que compren un producto, diagnosticar pacientes en base a síntomas similares a otros ya conocidos, o identificar correos electrónicos como spam o no spam.

10.2. Regresión

Para regresión, KNN predice un valor continuo promediando los valores de los vecinos más cercanos. Esto se puede aplicar, por ejemplo, para estimar el precio de una propiedad según características de casas similares, predecir la temperatura en una localidad basándose en estaciones cercanas, o calcular la demanda esperada de un producto a partir de comportamientos de clientes semejantes.

11. Optimizaciones y Variantes

Aunque KNN es simple, existen varias estrategias y variantes para mejorar su eficiencia y desempeño. Una optimización común es usar estructuras de datos como *KD-Trees* o *Ball Trees* que aceleran la búsqueda de vecinos, especialmente en datasets grandes o de alta dimensión. También se puede ponderar la influencia de los vecinos según la distancia, dando más peso a los más cercanos en lugar de tratarlos a todos por igual. Otra variante es *Weighted KNN*, que ajusta las predicciones en función de la proximidad. Para manejar mejor el ruido y outliers, se pueden combinar técnicas de filtrado de datos o ajustar dinámicamente el valor de k según la densidad local de puntos. Además, existen extensiones como KNN basado en densidad o con métricas adaptativas que ajustan la distancia según la distribución de los datos, y versiones aproximadas que reducen la complejidad computacional sin perder demasiada precisión.